

Teste de Arquitetura: Sistema de Simulação e Proposta de Empréstimos

Desenho da arquitetura e escolhas tecnológicas

A arquitetura pensada segue um modelo onde o cliente irá se comunicar através do frontend (WEB/APP) com os microserviços passando por um API Gateway e em seguida por um BFF(Backend for frontend).

A Api Gateway seria uma camada única de entrada para todas as requisições externas, com isso poderíamos obter as seguintes vantagens:

- **Segurança centralizada:** Pode aplicar autenticação, autorização e TLS em um ponto único antes que qualquer requisição alcance os sistemas internos.
- **Configuração de Rate limitint:** Evitando sobrecarga ao microserviço principal
- **Roteamento e agregação de chamadas:** Direciona requisições para diferentes serviços de forma transparente para o cliente.

Ao passar pela api gateway as requisições encontrariam o BFF antes de efetivamente alcançarem o microserviço de empréstimo (nosso core). O BFF atuaria como um intermediário entre o frontend e os microserviços sendo responsável por por exemplo:

- **Adaptação de payload:** Converte respostas dos microserviços em formatos otimizados para o consumo do frontend, evitando sobrecarga de dados.
- **Agregação de dados:** Junta informações de múltiplos microserviços em uma única resposta, reduzindo número de requisições no cliente.
- **Separação de responsabilidades:** Mantém a lógica de apresentação e transformação de dados fora dos microserviços core.
- **Versionamento e evolução independente:** Permite que mudanças no frontend não quebrem a compatibilidade com os microserviços.
- **Performance:** Reduzindo latência percebida pelo usuário, já que na arquitetura proposta ele utilizaria uma estratégia de cache para lidar com dados que fossem consultados repetidamente em um espaço de tempo configurável.

Ao passar pelo BFF a requisição de empréstimo finalmente alcançaria nosso microserviço principal (microserviço de empréstimos) que será o responsável pela lógica e orquestração do fluxo principal da aplicação, esse microserviço se comunicaria com alguns serviços externos (Como um motor de decisão e um motor de preços por exemplo) e por fim se comunicaria com um serviço externo para inclusão de proposta.

O microserviço de empréstimo faria uso de um banco de dados relacional, onde entendo que sua utilização seria importante para garantir a integridade dos dados aliado a boa performance, nesse nível poderiam ficar salvos, por exemplo, dados financeiros de clientes bem como um histórico de transações já realizadas pelo mesmo. Uma estrutura de banco de dados em cache também foi pensada nesse nível da aplicação, para poder gravar possíveis dados consultados com uma frequência alta.

Considerações sobre escalabilidade:

Penso que a arquitetura proposta, baseada em microserviços poderia ser escalada individualmente de acordo com a necessidade do negocio, a API Gateway por exemplo, poderia pode ser replicado em múltiplas instâncias para lidar com alto volume de requisições, o **BFF** também pode ser escalado de forma independente conforme a demanda de consumo do frontend, sem impactar diretamente os microserviços core assim como o microserviço de empréstimo também poderia ser escalado de forma individual.

Padrões de projeto e boas práticas:

Implementação de **Strategies** para variações de lógica de fluxo como por exemplo:

1. Fluxos de simulação com diferentes regras de acordo com o canal solicitante
2. Cálculos de taxas com variações dependendo do perfil do cliente / ou canal solicitante

Implementação de **estratégias de Retry** pra consumo de serviços externos, tornando assim o código menos suscetível a erros por dependencia externa.

Apesar do fluxo atual estar desenhado em alto nível, sem uma especificação acerca de uma comunicação síncrona e/ou assíncrona acredito que por se tratar de uma arquitetura voltada ao mercado financeiro e que requer comunicação com motores de decisão e pricing por exemplo, é provável que, em algum momento, a comunicação assíncrona seja necessária. Nesse contexto, a adoção do **Event-Driven Pattern** (arquitetura orientada a eventos) seria um exemplo relevante, permitindo criar processamentos desacoplados e mais flexíveis dentro do fluxo da aplicação.

Além disso, **automação de testes** é essencial para garantir confiança durante o processo de homologação de novas features, assim como a utilização de **ferramentas de qualidade de código** para manter padrões elevados. Nesse sentido, a aplicação de princípios de **Clean Code** é fortemente recomendada para manter o código legível, modular, de fácil manutenção e com baixo acoplamento, favorecendo a evolução da aplicação ao longo do tempo.

Por fim, a **definição e monitoramento de métricas** é recomendada para apoiar processos de debug e monitoramento em produção, contribuindo para a observabilidade e manutenção contínua do sistema.

Autenticação e autorização:

O controle de acesso será centralizado na **API Gateway**, que atuará como ponto único para validação de credenciais e aplicação de políticas de segurança. A autenticação será baseada em **tokens JWT (JSON Web Token)**, permitindo que cada requisição carregue de forma segura as informações de identidade e permissões do usuário.

Esse modelo dispensa a necessidade de consultas constantes a um servidor de autenticação, reduzindo a latência e melhorando a escalabilidade. A **API Gateway** será responsável por validar a assinatura e a integridade do token antes de encaminhar as requisições aos serviços internos, garantindo que apenas clientes devidamente autenticados e autorizados acessem os recursos da aplicação.

API Design:

POST /api/emprestimos/solicitar-simulacao

Endpoint responsável pelo recebimento dos dados necessários para realização da simulação;

Request:

```
{
  "cpf": "000.111.222-33",
  "nome": "José de Almeida",
  "dataNascimento": "1988-12-17",
  "rendaMensal": 8000,
  "valorSolicitado": 12000,
  "prazo": 36
}
```

Response:

```
{
```

```
    "mensagem": "Solicitação de simulação recebida com sucesso.",
    "idSimulacao": "123456"
}
```

GET /api/emprestimos/simulacoes/{idSimulacao}

Endpoint responsável por buscar os dados da simulação com o id informando no path.

Request:

/api/emprestimos/simulacoes/123456

Response:

```
{
  "idSimulacao": "123456",
  "valorSolicitado": "12000",
  "simulacaoAprovada": true
  "dataBase": "2025-08-10"
  "condicoesEmprestimo": [
    {
      "id": 1,
      "valorEmprestimo": 12000,
      "prazoMeses": 24,
      "valorTotalAPagar": 12374.48,
      "parcelaMensal": 288.18,
      "totalJurosPagos": 374.48,
      "jurosAnual": 0.03,
      "jurosMensal": 0.002
    },
    {
      "id": 2,
      "valorEmprestimo": 12000,
      "prazoMeses": 36,
      "valorTotalAPagar": 12674.48,
      "parcelaMensal": 308.18,
      "totalJurosPagos": 674.48,
      "jurosAnual": 0.03,
      "jurosMensal": 0.002
    },
  ]
  motivoRecusa: null (caso simulacaoAprovada fosse false esse objeto seria preenchido)
}
```

GET /api/emprestimos/simulacoes/{cpf}

Endpoint responsável por retornar uma lista de simulacoes pelo cpf do cliente.

Request:

/api/emprestimos/simulacoes/12345678900

Response:

```

{
  "simulacoes": [
    {
      "idSimulacao": "123456",
      "valorSolicitado": 12000,
      "simulacaoAprovada": true,
      "dataBase": "2025-08-10",
      "condicoesEmprestimo": {
        "id": 1,
        "valorEmprestimo": 12000,
        "prazoMeses": 24,
        "valorTotalAPagar": 12374.48,
        "parcelaMensal": 288.18,
        "totalJurosPagos": 374.48,
        "jurosAnual": 0.03,
        "jurosMensal": 0.002
      },
      "motivoRecusa": null
    },
    {
      "idSimulacao": "789012",
      "valorSolicitado": 8000,
      "simulacaoAprovada": false,
      "dataBase": "2024-12-20",
      "condicoesEmprestimo": null,
      "motivoRecusa": {
        "codigo": "RENDA_INSUFICIENTE",
        "descricao": "A renda informada não atende aos critérios mínimos para aprovação."
      }
    }
  ]
}

```

POST /api/emprestimos/gerar-proposta

Endpoint responsável por gerar uma proposta de acordo com a simulação e condição de empréstimo escolhida pelo cliente.

Request:

```

{
  "idSimulacao": "123456",
  "idCondicaoEmprestimo": 1
}

```

```
}
```

Response:

```
{
  "idProposta": "12345",
  "idSimulacao": "123456",
  "valorEmprestimo": 12000,
  "prazoMeses": 24,
  "valorTotalAPagar": 12374.48,
  "parcelaMensal": 288.18,
  "totalJurosPagos": 374.48,
  "jurosAnual": 0.03,
  "jurosMensal": 0.002,
  "dataProposta": "2025-08-11",
}
```

GET /api/emprestimos/proposta/{idProposta}

Endpoint responsável por trazer os detalhes de uma proposta.

Request:

/api/emprestimos/proposta/12345

Response:

```
{
  "idProposta": "12345",
  "statusProposta": "EM_ANDAMENTO",
  "condicaoEmprestimo": {
    "valorEmprestimo": 12000,
    "prazoMeses": 24,
    "valorTotalAPagar": 12374.48,
    "parcelaMensal": 288.18,
    "totalJurosPagos": 374.48,
    "jurosAnual": 0.03,
    "jurosMensal": 0.002
  },
  "cliente": {
    "cpf": "000.111.222-33",
    "nome": "José de Almeida",
    "dataNascimento": "1988-12-17",
  },
  "dataProposta": "2025-08-11",
  "erros": null
}
```